

Universidad Nacional de Rosario
Facultad de Ciencias Exactas, Ingeniería y Agrimensura
Tecnicatura Universitaria en Inteligencia Artificial

Procesamiento de Imágenes I

Trabajo Práctico N° 1

Ecualización local de histograma y validación de planillas de calificaciones

Integrantes

Valentín Cura
Bautista Cortinas
Franco Maragliano

2026 - 2° semestre

1. Introducción

En este trabajo resolvimos los dos problemas del enunciado. En el primero implementamos la ecualización local de histograma y la usamos para encontrar los detalles ocultos de una imagen. En el segundo hicimos un programa que revisa planillas de calificaciones (que vienen como imágenes) y controla que cada campo cumpla las reglas del enunciado; además genera una imagen con los alumnos que no aprobaron y un archivo CSV con los resultados.

Usamos lo que vimos en clase: OpenCV, numpy y matplotlib, umbralado, suma de píxeles por filas y columnas, componentes conectadas, contornos e histogramas, y la función `imshow` que dio la cátedra. Para cada problema contamos qué se pide, qué dificultades tuvimos, cómo lo resolvimos, mostramos capturas de pasos intermedios y escribimos las conclusiones.

2. Problema 1: ecualización local de histograma

2.1 Qué se pide

La ecualización global de histograma calcula una sola transformación con todos los píxeles de la imagen. Si hay zonas con detalles de intensidad parecida a la de su fondo, no sirve, porque el histograma global lo dominan las partes más grandes de la imagen. En la ecualización local se usa una ventana de $M \times N$ píxeles que se va moviendo de a un píxel. En cada posición se calcula el histograma de la ventana, se arma la transformación de la ecualización y se usa solamente para el píxel que está en el centro.

El enunciado pide: (a) hacer una función que reciba la imagen y el tamaño de la ventana, (b) usarla con la imagen de la figura 1 e informar qué detalles ocultos hay, y (c) analizar cómo influye el tamaño de la ventana.

2.2 Dificultades

Bordes. Los píxeles de los bordes no tienen una ventana completa. Agregamos un borde con `cv2.copyMakeBorder` y `BORDER_REPLICATE`. Para que también ande con ventanas de tamaño par, arriba y a la izquierda ponemos $M/2$ y $N/2$ píxeles y el resto va abajo y a la derecha.

Tiempo de cálculo. Hay que hacer una ventana por cada píxel, así que el costo es alto y crece con el tamaño de la ventana ($\text{alto} \times \text{ancho} \times M \times N$). Para ahorrar trabajo no calculamos la transformación entera de 256 valores: como solo se usa el valor del píxel central, alcanza con evaluar la distribución acumulada en ese nivel. El resultado es el mismo.

Tipo de dato. La imagen es `uint8`. Hicimos las cuentas con números reales y recién al final redondeamos y pasamos a `uint8`.

Detalles casi iguales al fondo. Para entender por qué la ecualización global no los muestra, miramos el histograma (sección 2.4).

2.3 Cómo lo resolvimos

Para un píxel de intensidad r , la transformación es $s = (L - 1) \cdot \text{CDF}(r)$, con $L = 256$ niveles y $\text{CDF}(r)$ la fracción de píxeles de la ventana que son menores o iguales que r . La función `ecualizacion_local_histograma(img, ventana)` hace lo siguiente:

1. Agregar bordes a la imagen (BORDER_REPLICATE)
2. Para cada píxel (i, j):
 - a. tomar la ventana MxN centrada en (i, j)
 - b. calcular su histograma (np.bincount, 256 niveles)
 - c. calcular la distribución acumulada dividida por MxN
 - d. nuevo valor = round(255 * CDF[valor original del píxel])
3. Devolver la imagen resultante (uint8, mismo tamaño)

La ventana se puede pasar como un entero (cuadrada) o como una tupla (M, N).

2.4 Pasos intermedios

La imagen mide 256×256. Su histograma tiene solo dos grupos de valores: de 0 a 11 (los cuadrados oscuros y lo que hay adentro) y de 226 a 228 (el fondo). Dentro de cada cuadrado, los detalles se diferencian del resto del cuadrado en apenas uno o dos niveles. Con la ecualización global esa diferencia no se nota, porque casi todos los píxeles caen en los dos grupos grandes.

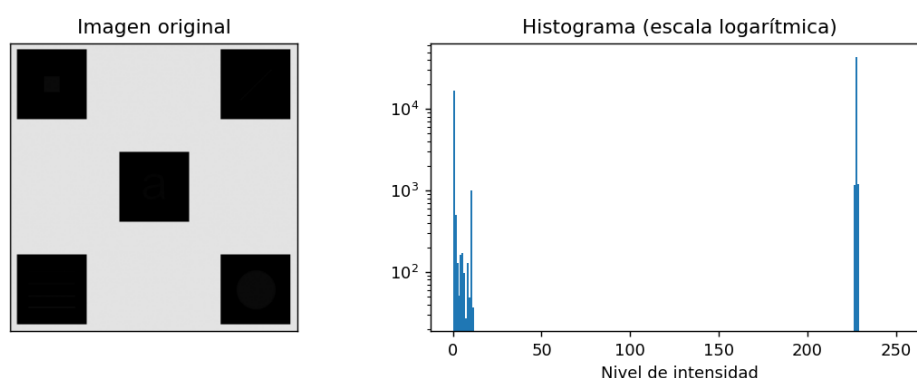


Figura 1: imagen original e histograma (escala logarítmica).

2.5 Detalles ocultos (punto b)

En la figura 2 se compara la imagen original, la ecualización global y la local con distintos tamaños de ventana. La global no muestra nada nuevo; la local deja ver un objeto distinto en cada cuadrado:

Zona	Detalle oculto
Superior izquierda	Un cuadrado pequeño
Superior derecha	Una línea diagonal
Centro	La letra "a"
Inferior izquierda	Unas cinco líneas horizontales
Inferior derecha	Un círculo

Ecualización local de histograma - Influencia del tamaño de ventana

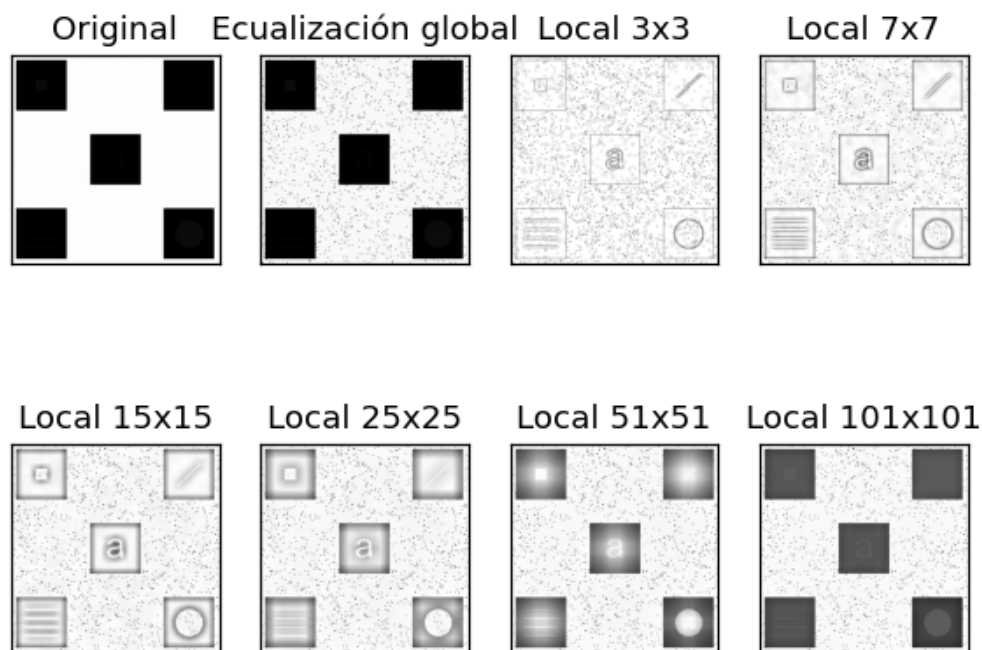


Figura 2: original, ecualización global y ecualización local con ventanas de 3, 7, 15, 25, 51 y 101.

También se ven puntos sueltos en el fondo. Es porque el fondo no es del todo uniforme (varía entre 226 y 228) y la ecualización local agranda esas diferencias mínimas.

2.6 Influencia del tamaño de la ventana (punto c)

Ventana	Qué se ve	Por qué
3×3	Se ven los contornos de los objetos y de los cuadrados, pero hay mucho ruido en el fondo	La ventana ve muy poco y agranda cualquier variación
7×7	Formas bien definidas; el ruido de fondo sigue	Tamaño parecido al de los trazos
15×15	Es el resultado más limpio y fácil de leer	Buen equilibrio entre detalle y ruido
25×25	Se distinguen los objetos, pero aparece un halo brillante en los bordes de los cuadrados	La ventana empieza a agarrar fondo claro de afuera
51×51	Degradado del borde al centro y detalles más borrosos	Ver explicación abajo; se parece cada vez más a la global
101×101	Casi no se distinguen los detalles	La ventana es más grande que el cuadrado y se pierde lo local

El degradado de 51×51 se explica con la fórmula: el nuevo valor es 255 por la fracción de píxeles de la ventana que son menores o iguales que el del centro. En el medio de un cuadrado la ventana tiene solo valores del cuadrado, esa fracción es alta y el píxel queda brillante. Cerca del borde la ventana incluye fondo claro (valores más altos), la fracción baja y el píxel se oscurece.

2.7 Conclusiones del Problema 1

La ecualización local muestra objetos que la global no puede mostrar, porque usa el contexto de cada píxel. El tamaño de la ventana es lo más importante: tiene que ser parecido al tamaño de los detalles que se buscan. Si es muy chica aparece mucho ruido y si es muy grande se pierde lo local. En esta imagen los mejores resultados fueron con ventanas de entre 7 y 25, y el más claro con 15×15 . La desventaja es el tiempo de cálculo, que crece con la cantidad de píxeles y con el área de la ventana.

3. Problema 2: validación de planillas de calificaciones

3.1 Qué se pide

Cada planilla es una imagen con una tabla de 20 registros y siete columnas: Nro., Legajo, Nombre y Apellido, Parcial 1, Parcial 2, Parcial 3 y Condición Final. Hay que verificar que cada campo cumpla estas reglas (solo se controla el formato, no si el dato es válido):

Campo	Regla
Legajo	8 caracteres en total, formando una sola palabra
Nombre y apellido	Mínimo dos palabras y no más de 12 caracteres en total
Parcial 1, 2 y 3	1 o 2 caracteres consecutivos (sin espacios)
Condición Final	Un solo carácter

El programa tiene que: (a) mostrar por terminal qué campos están OK y cuáles MAL en cada registro; (b) generar una imagen con los alumnos que no aprobaron (Condición Final "L" o "R"), usando solo los registros bien cargados y marcando distinto a los que recuperan ("R") y a los libres ("L"); (c) generar un CSV con los resultados; y (d) aplicarse de forma cíclica a las cuatro planillas.

3.2 Dificultades

Las planillas son de distinto tamaño. Las cuatro imágenes tienen tamaños distintos y las columnas tienen anchos distintos, así que no se pueden usar coordenadas fijas. Detectamos las líneas de la tabla con la suma de píxeles oscuros por filas y por columnas, con umbrales relativos al máximo (60 % para las horizontales y 50 % para las verticales).

Encabezado de dos niveles. "Notas" agrupa tres columnas y la línea que queda debajo no cruza todo el ancho, por eso el umbral del 60 % la deja afuera. Quedan 22 líneas horizontales (borde de arriba, fin del encabezado y una por registro) y 8 verticales. Si el programa no encuentra esa cantidad, avisa con un error.

El banner de arriba. Los logos tienen píxeles oscuros que se podrían confundir con líneas. Umbralamos con un valor bajo (menos de 100), que deja solo los píxeles casi negros.

Restos de líneas dentro de las celdas. Para evitar componentes conectadas falsas recortamos cada celda 2 píxeles hacia adentro. No usamos el filtro por área que sugiere la ayuda: el guion tiene un área de solo 5 píxeles y el punto de "1.0" es todavía más chico, así que un filtro por área los borraría y "1.0" se leería como "10".

Espacios. La fuente es de ancho fijo, entonces la distancia entre los centros de dos caracteres seguidos es siempre la misma (el "paso", cerca de 10 píxeles, que estimamos con la mediana de cada planilla). Si la distancia es de más o menos dos pasos, hay un espacio.

La Ñ. Su tilde es una componente conectada aparte. Unimos las componentes que se superponen horizontalmente en más de la mitad del ancho de la más angosta.

Leer la Condición Final sin reconocedor de caracteres. Clasificamos las letras A, L y R por su forma (ver 3.4).

3.3 Cómo lo resolvimos

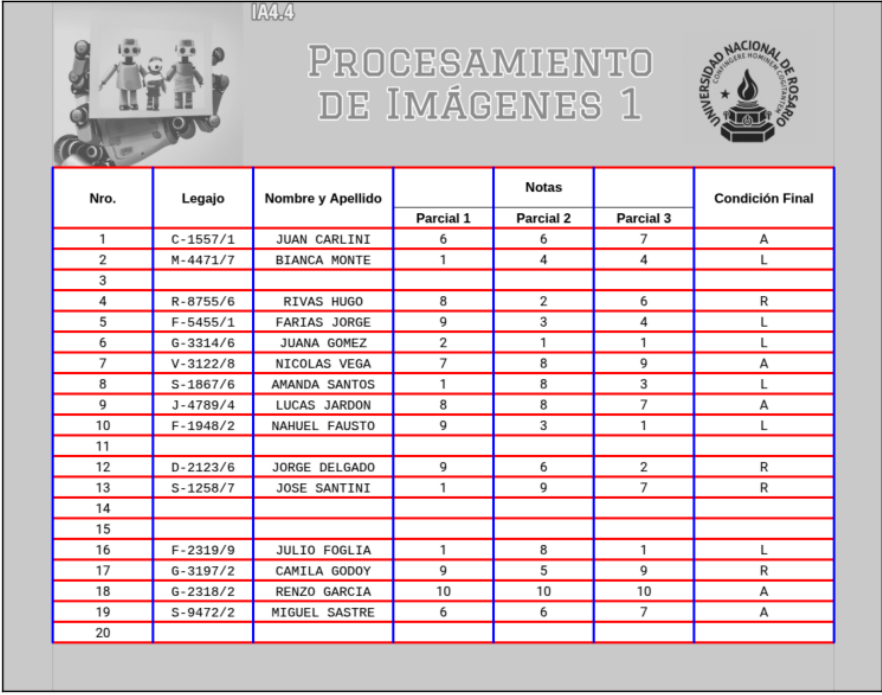
La función `validar_planilla` hace estos pasos con cada imagen:

Etapa	Qué hace
1. Lectura	Lee la imagen en escala de grises con <code>cv2.imread</code>
2. Líneas	Umbralado (<code>img < 100</code>), suma por filas y columnas con <code>np.sum</code> y búsqueda de tramos seguidos (las líneas pueden tener más de un píxel de ancho)
3. Celdas	Con las líneas se arman las 6 celdas de datos de cada uno de los 20 registros (la columna Nro. no se valida)
4. Caracteres	Binariza la celda (<code>img < 128</code>) y usa <code>cv2.connectedComponentsWithStats</code> ; ordena las componentes de izquierda a derecha
5. Largo y palabras	Con las distancias entre centros cuenta caracteres, espacios y palabras
6. Validación	Aplica las reglas de la tabla de 3.1. Una celda vacía tiene 0 caracteres, así que da MAL
7. Salidas	Mensajes por terminal, imagen de no aprobados y CSV (UTF-8, columnas ID, Legajo, Nombre y Apellido, Parcial 1, Parcial 2, Parcial 3 y Condición Final, con valores OK o MAL)

3.4 Pasos intermedios

La figura 3 muestra las líneas detectadas en la planilla 1 y la figura 4 las sumas por filas y por columnas con su umbral. Las líneas de la tabla son picos mucho más altos que el resto, por eso se separan fácil con un umbral.

Planilla 1 - Lineas detectadas



Nro.	Legajo	Nombre y Apellido	Notas			Condición Final
			Parcial 1	Parcial 2	Parcial 3	
1	C-1557/1	JUAN CARLINI	6	6	7	A
2	M-4471/7	BIANCA MONTE	1	4	4	L
3						
4	R-8755/6	RIVAS HUGO	8	2	6	R
5	F-5455/1	FARIAS JORGE	9	3	4	L
6	G-3314/6	JUANA GOMEZ	2	1	1	L
7	V-3122/8	NICOLAS VEGA	7	8	9	A
8	S-1867/6	AMANDA SANTOS	1	8	3	L
9	J-4789/4	LUCAS JARDON	8	8	7	A
10	F-1948/2	NAHUEL FAUSTO	9	3	1	L
11						
12	D-2123/6	JORGE DELGADO	9	6	2	R
13	S-1258/7	JOSE SANTINI	1	9	7	R
14						
15						
16	F-2319/9	JULIO FOGLIA	1	8	1	L
17	G-3197/2	CAMILA GODOY	9	5	9	R
18	G-2318/2	RENZO GARCIA	10	10	10	A
19	S-9472/2	MIGUEL SASTRE	6	6	7	A
20						

Figura 3: líneas horizontales (rojo) y verticales (azul) detectadas en la planilla 1.

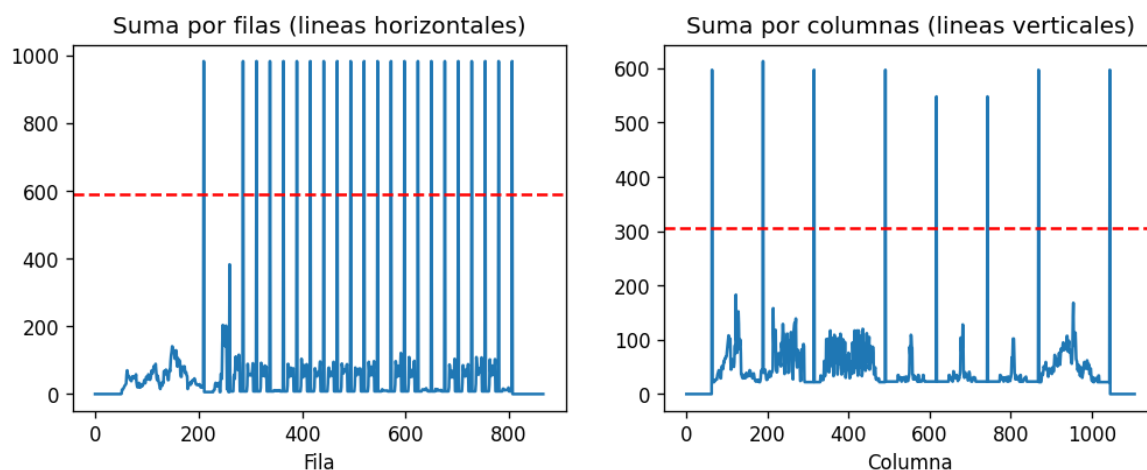


Figura 4: suma de píxeles oscuros por fila y por columna. La línea punteada es el umbral.

La figura 5 muestra los caracteres detectados en los campos del primer registro de la planilla 1. Cada rectángulo rojo es una componente conectada. En el legajo se cuentan 8 caracteres seguidos, así que cumple la regla.

Planilla 1 - Registro 1: caracteres detectados

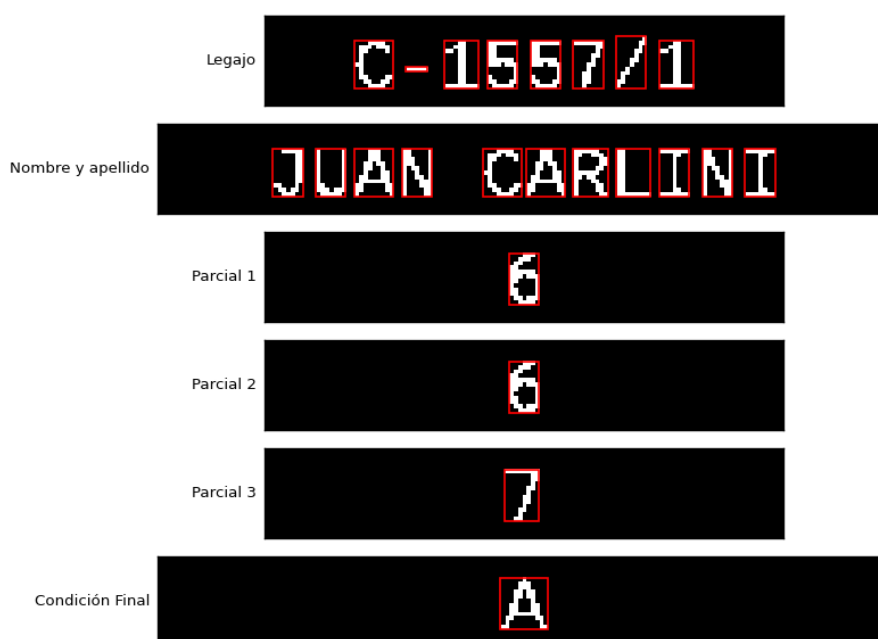


Figura 5: caracteres detectados en los seis campos del registro 1 de la planilla 1.

La figura 6 muestra casos que ponen a prueba las reglas: un legajo correcto (8 caracteres, 1 palabra); un legajo con un espacio antes del último carácter (9 caracteres, 2 palabras); un legajo de 8 caracteres partido en 2 palabras por un espacio; la nota "1.0", que da tres caracteres porque el punto es una componente; un nombre de tres palabras y 16 letras; y un nombre de una sola palabra.

Planilla 1 - Registro 1 - Legajo: 8 caracteres, 1 palabra(s) → OK



Planilla 2 - Registro 19 - Legajo: 9 caracteres, 2 palabra(s) → MAL



Planilla 3 - Registro 9 - Legajo: 8 caracteres, 2 palabra(s) → MAL



Planilla 3 - Registro 15 - Parcial 2: 3 caracteres, 1 palabra(s) → MAL



Planilla 2 - Registro 13 - Nombre y apellido: 16 caracteres, 3 palabra(s) → MAL



Planilla 3 - Registro 8 - Nombre y apellido: 12 caracteres, 1 palabra(s) → MAL



Figura 6: casos particulares y su resultado.

Para la Condición Final usamos `cv2.findContours` con `RETR_CCOMP` y contamos los agujeros de la letra: la "L" no tiene ninguno y la "A" y la "R" tienen uno. Para separar la "A" de la "R" medimos cuánta tinta hay en las dos primeras columnas: la "R" tiene el palo vertical a la izquierda (valor alto) y la "A" solo el pie de su pata (valor bajo).

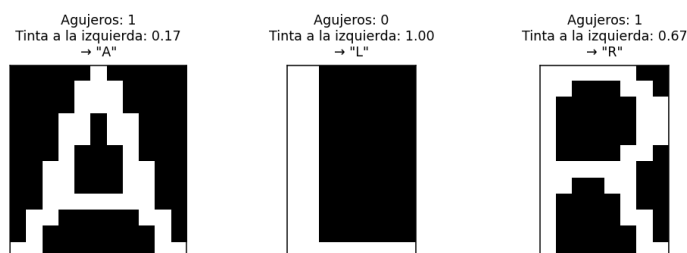


Figura 7: clasificación de las letras A, L y R.

3.5 Resultados

Esta tabla resume lo que dio el programa en las cuatro planillas. Las filas sin ningún carácter se cuentan como registros con todos los campos en MAL.

Planilla	Registros correctos	Registros vacíos	MAL por campo (Legajo / Nombre / P1 / P2 / P3 / Cond.)	No aprobaron (n° de registro)
1	15 de 20	5	5 / 5 / 5 / 5 / 5 / 5	2 L, 4 R, 5 L, 6 L, 8 L, 10 L, 12 R, 13 R, 16 L, 17 R
2	3 de 20	3	8 / 9 / 7 / 4 / 7 / 8	2 L, 5 L, 8 L
3	0 de 20	3	10 / 9 / 7 / 5 / 8 / 8	Ninguno
4	6 de 20	3	6 / 7 / 4 / 5 / 5 / 6	7 L, 12 R, 17 R, 19 L

Alumnos que no aprobaron

02	BIANCA MONTE	LIBRE
04	RIVAS HUGO	RECUPERA
05	FARIAS JORGE	LIBRE
06	JUANA GOMEZ	LIBRE
08	AMANDA SANTOS	LIBRE
10	NAHUEL FAUSTO	LIBRE
12	JORGE DELGADO	RECUPERA
13	JOSE SANTINI	RECUPERA
16	JULIO FOGLIA	LIBRE
17	CAMILA GODOY	RECUPERA

Figura 8: imagen de salida de la planilla 1. Naranja: recupera (R). Rojo: libre (L).

3.6 Decisiones que tomamos y limitaciones

El enunciado deja algunos puntos abiertos. Esto es lo que decidimos:

Espacios en el nombre. No los contamos para el máximo de 12 caracteres (se cambia con la constante `CONTAR_ESPACIOS_EN_NOMBRE`). Lo decidimos porque en la planilla 1, que parece la correcta, cuatro nombres tienen exactamente 12 letras. Si los espacios cuentan, los registros correctos pasan a ser 11, 2, 0 y 5 en las planillas 1, 2, 3 y 4.

Filas vacías. Las tomamos como registros con todos los campos en MAL, porque un campo vacío no cumple los mínimos de caracteres.

Salidas. Generamos una imagen y un CSV por planilla, porque el programa recibe una sola planilla como entrada.

Limitaciones. Esperamos una tabla de 20 registros y 7 columnas (si no coincide, el programa avisa con un error). La Condición Final se lee por la forma de las letras A, L y R en esta fuente; cualquier otra letra no se reconoce y el registro no entra en la imagen de salida. Tampoco hay un reconocimiento general de caracteres.

3.7 Conclusiones del Problema 2

Pudimos resolverlo con lo visto en clase: líneas por proyecciones, componentes conectadas para separar los caracteres y contornos para distinguir las letras de la Condición Final. Lo que hizo que ande con planillas de distinto tamaño fue usar umbrales relativos en lugar de coordenadas fijas, y que la fuente sea de ancho fijo nos permitió contar espacios sin reconocer caracteres. No filtrar por área nos permitió detectar casos como "1.0". Las limitaciones son que depende de una tabla con estructura conocida y de una fuente de ancho fijo.

4. Conclusiones generales

En los dos problemas funcionó mejor trabajar con información local y con parámetros relativos a la imagen que usar operaciones globales o valores fijos. En el Problema 1 la ventana local muestra objetos que la ecualización global no deja ver, y en el Problema 2 las proyecciones por filas y columnas se adaptan a planillas de distintos tamaños. Armamos funciones que se pueden reutilizar y dejamos anotadas las decisiones donde el enunciado no es claro.

5. Cómo ejecutar

El README.md del repositorio tiene las versiones de las librerías (Python 3.12.3, numpy 2.4.4, opencv-python 4.13.0 y matplotlib 3.10.8), dónde poner las imágenes de entrada y cómo ejecutar cada script. Los resultados se guardan en una carpeta salidas que se crea sola.